Adaptive Hybrid SAT-HCP

Từ nhập môn SAT đến thiết kế một phương pháp tích hợp

Tài liệu này giải thích bài toán chu trình Hamilton, SAT, CEGAR, cut-set và user propagator bằng trực giác trước, công thức sau. Phần cuối trình bày một phương pháp duy nhất có khả năng học và tận dụng các kỹ thuật tốt nhất từ những baseline hiện có.

Dự án HCP-CEGAR

Phiên bản thiết kế, tháng 8 năm 2026

Cách đọc tài liệu

Tài liệu được viết cho cả người chưa từng học SAT. Người đọc không cần biết logic mệnh đề, solver hay CEGAR trước đó.

- Nếu mới bắt đầu, hãy đọc tuần tự từ Chương 1 đến Chương 4.
- Nếu đã biết SAT, có thể bắt đầu từ Chương 4 về CEGAR và cut-set.
- Nếu quan tâm trực tiếp đến phương pháp mới, đọc Chương 6 đến Chương 10.
- Các hộp màu xanh giải thích trực giác; hộp cam nêu rủi ro; hộp viền đậm chốt kết luận.

Các con số trong Chương 5 đến từ dữ liệu đã có. Kiến trúc Adaptive Hybrid SAT-HCP trong các chương sau là một *đề xuất nghiên cứu*: các module thành phần đã tồn tại hoặc đã có bằng chứng riêng, nhưng lợi ích của toàn bộ hệ tích hợp vẫn phải được kiểm nghiệm bằng một benchmark công bằng.

Mục lục

Cách đọc tài liệu i

1. Bài toán chu trình Hamilton là gì? 1
- 1.1. Đồ thị: thành phố và con đường . . . 1
- 1.2. Tại sao bài toán khó? . . . 1

2. SAT cho người mới bắt đầu 2
- 2.1. SAT là bài toán những công tắc đúng hoặc sai . . . 2
- 2.2. Literal, clause và công thức CNF . . . 2
- 2.3. SAT solver làm việc như thế nào? . . . 2
- 2.4. SAT, UNSAT và UNKNOWN . . . 3

3. Biến HCP thành SAT 4
- 3.1. Ràng buộc bậc: mỗi đỉnh chọn đúng hai cạnh . . . 4
- 3.2. Vấn đề cycle cover . . . 4
- 3.3. Tại sao không thêm mọi ràng buộc ngay từ đầu? . . . 4

4. CEGAR và cut-set bằng trực giác 5
- 4.1. CEGAR là vòng lặp "đoán, kiểm tra, sửa" . . . 5
- 4.2. Blocking subcycle và cut-set khác nhau thế nào? . . . 5
- 4.3. 2-opt và 3-opt: sửa lời giải thay vì bỏ đi . . . 5

5. Ta học được gì từ các baseline? 7
- 5.1. Phân biệt cấu hình đơn, portfolio và VBS . . . 7
- 5.2. Bức tranh dữ liệu hiện có . . . 7
- 5.3. Bài học cụ thể từ từng baseline . . . 8

6. Phương pháp đề xuất: Adaptive Hybrid SAT-HCP 9
- 6.1. Một phương pháp, không phải một túi solver . . . 9
- 6.2. Năm nguyên tắc thiết kế . . . 9
- 6.3. Kiến trúc tổng thể . . . 9
- 6.4. Phase A: CEGAR có inprocessing . . . 9
- 6.5. Phase B: SAT-guided repair . . . 10
- 6.6. Phase C: prevent-subcycle user propagation . . . 10
- 6.7. Vì sao cần rebuild giữa Phase A và Phase C? . . . 10

7. Hybrid Controller quyết định như thế nào? 11
- 7.1. Telemetry tối thiểu . . . 11
- 7.2. Controller luật trước, học máy sau . . . 11
- 7.3. Phân bổ ngân sách . . . 11

8. Các hướng brainstorming có tiềm năng 13
- 8.1. Encoding lai theo từng đỉnh . . . 13
- 8.2. GraphCutPool độc lập encoding . . . 13
- 8.3. Phase hints từ cycle cover tốt nhất . . . 13
- 8.4. Tổng quát hóa cut theo xung đột . . . 13
- 8.5. Học controller bằng dữ liệu . . . 13

9. Thiết kế thực nghiệm công bằng **15**
9.1. Câu hỏi nghiên cứu . . . 15
9.2. Ba mức kết quả phải báo cáo riêng . . . 15
9.3. Tập đánh giá trọng tâm . . . 15
9.4. Chỉ số đánh giá . . . 15
9.5. Ablation bắt buộc . . . 16

10. Lộ trình triển khai **17**
10.1. Giai đoạn 0: hoàn thiện bằng chứng . . . 17
10.2. Giai đoạn 1: GraphCutPool và telemetry . . . 17
10.3. Giai đoạn 2: Hybrid Controller luật . . . 17
10.4. Giai đoạn 3: tích hợp kỹ thuật HCup . . . 17
10.5. Giai đoạn 4: cải tiến mới và học selector . . . 18

11. Pseudocode dễ đọc **19**

12. Câu hỏi thường gặp **20**

A. Bảng thuật ngữ **21**

B. Tái tạo báo cáo coverage và benchmark **22**

C. Tài liệu tham khảo **23**

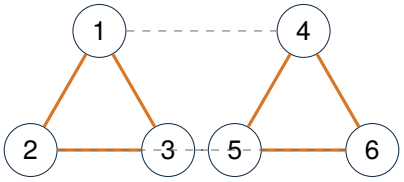
1. Bài toán chu trình Hamilton là gì?

1.1. Đồ thị: thành phố và con đường

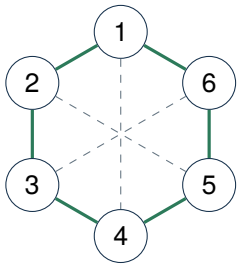
Một *đồ thị* gồm các đỉnh và các cạnh. Có thể hình dung đỉnh là thành phố, cạnh là con đường nối hai thành phố. Trong tài liệu này, ta xét đồ thị vô hướng: nếu đi được từ u đến v thì cũng đi được từ v đến u.

Một *chu trình Hamilton* là một chuyến đi:

1. đi qua mọi đỉnh đúng một lần;
2. sau cùng quay về đỉnh xuất phát;
3. chỉ sử dụng những cạnh có trong đồ thị.



(a) Mỗi đỉnh có bậc 2 trong lời giải, nhưng có hai chu trình rời nhau.



(b) Một chu trình duy nhất đi qua cả sáu đỉnh: đây là chu trình Hamilton.

Hình 1.1.: Khác biệt giữa một cycle cover và một chu trình Hamilton.

Trực giác

Điều khó không phải chỉ là chọn hai cạnh tại mỗi đỉnh. Điều khó là buộc tất cả các cạnh được chọn phải nối thành *một* vòng duy nhất. Nhiều vòng nhỏ như hình bên trái là một lời giải gần đúng nhưng chưa hợp lệ.

1.2. Tại sao bài toán khó?

Với một đồ thị nhỏ, ta có thể thử từng đường đi. Khi số đỉnh tăng, số khả năng tăng quá nhanh. hcp là bài toán NP-complete: chưa có thuật toán tổng quát nào được biết có thể giải nhanh mọi trường hợp.

Điều này không có nghĩa mọi instance đều khó. Nhiều đồ thị lớn được giải trong tích tắc, trong khi một số đồ thị nhỏ hơn có cấu trúc đối xứng hoặc nhiều lựa chọn gần giống nhau lại làm solver mất hàng chục phút.

2. SAT cho người mới bắt đầu

2.1. SAT là bài toán những công tắc đúng hoặc sai

Trong SAT, mỗi biến chỉ có hai giá trị:

$$x = \text{Đúng} \qquad \text{hoặc} \qquad x = \text{Sai}.$$

Với HCP, ta tạo một biến x_{uv} cho mỗi cạnh (u, v):

$$x_{uv} = \text{Đúng} \iff \text{cạnh } (u, v) \text{ được chọn vào chu trình.}$$

SAT solver có nhiệm vụ bật hoặc tắt hàng nghìn, thậm chí hàng triệu công tắc sao cho mọi quy tắc đều được thỏa mãn.

2.2. Literal, clause và công thức CNF

Một *literal* là một biến hoặc phủ định của biến:

$$x \quad \text{hoặc} \quad \neg x.$$

Một *clause* là phép "hoặc". Clause

$$(x_{12} \vee x_{13})$$

nói rằng phải chọn ít nhất một trong hai cạnh $(1, 2)$ và $(1, 3)$.

Một công thức CNF là phép "và" của nhiều clause:

$$(x_{12} \vee x_{13}) \wedge (\neg x_{12} \vee x_{14}) \wedge (\neg x_{13} \vee \neg x_{14}).$$

Giả sử x_{12} được bật. Clause thứ hai buộc x_{14} cũng bật. Khi đó clause thứ ba buộc x_{13} tắt. SAT solver thực hiện hàng loạt suy luận kiểu này, nhưng ở quy mô rất lớn và có cơ chế quay lui khi đi vào ngõ cụt.

2.3. SAT solver làm việc như thế nào?

Một solver hiện đại như CaDiCaL có thể được hình dung bằng bốn thao tác:

1. **Quyết định:** thử gán một biến là đúng hoặc sai.
2. **Lan truyền:** từ các clause hiện có, suy ra những biến bắt buộc.

3. **Phát hiện xung đột:** nhận ra rằng một nhánh lựa chọn không thể thỏa mãn.
4. **Học clause và quay lui:** ghi nhớ nguyên nhân xung đột để không lặp lại sai lầm tương tự.

SAT solver không chỉ "thử hết"

Điểm mạnh của SAT solver là học từ thất bại. Mỗi xung đột có thể tạo ra một quy tắc mới, loại bỏ cả một vùng lớn của không gian tìm kiếm thay vì chỉ loại một nghiệm.

2.4. SAT, UNSAT và UNKNOWN

Kết quả	Ý nghĩa
SAT	Tìm được một phép gán thỏa mọi clause. Với HCP, vẫn cần kiểm tra tour đầu ra.
UNSAT	Chứng minh không có phép gán nào thỏa công thức.
UNKNOWN/TIMEOUT	Chưa giải xong trong giới hạn tài nguyên; không được hiểu là UNSAT.

3. Biến HCP thành SAT

3.1. Ràng buộc bậc: mỗi đỉnh chọn đúng hai cạnh

Trong một chu trình Hamilton vô hướng, mỗi đỉnh có đúng hai cạnh được chọn: một cạnh để đi vào và một cạnh để đi ra. Với đỉnh v, ta cần:

$$\sum_{e \in \delta(v)} x_e = 2,$$

trong đó $\delta(v)$ là tập cạnh kề với v.

Máy SAT không hiểu trực tiếp phép cộng này, nên ta phải dịch nó thành các clause. Có nhiều cách dịch:

- **Pairwise/Binomial:** dễ hiểu, tốt khi số cạnh kề ít, nhưng sinh nhiều clause khi bậc lớn.
- **Sinz sequential counter:** thêm các biến phụ theo kiểu bộ đếm tuần tự; thường gọn và lan truyền tốt.
- **Commander:** chia biến thành nhóm rồi dùng biến chỉ huy; hữu ích cho tập biến lớn.
- **Adder/CRT:** mô phỏng phép cộng hoặc thứ tự bằng mạch logic; có ưu điểm ở một số cấu trúc nhưng không phải lúc nào cũng tốt nhất.

3.2. Vấn đề cycle cover

Chỉ ràng buộc "mỗi đỉnh đúng hai cạnh" chưa đủ. SAT solver có thể trả về nhiều chu trình nhỏ rời nhau như Hình 1.1. Kết quả đó gọi là một *cycle cover*.

Ta cần thêm ràng buộc kết nối toàn cục: không được tồn tại một tập đỉnh S bị cô lập khỏi phần còn lại của chu trình.

3.3. Tại sao không thêm mọi ràng buộc ngay từ đầu?

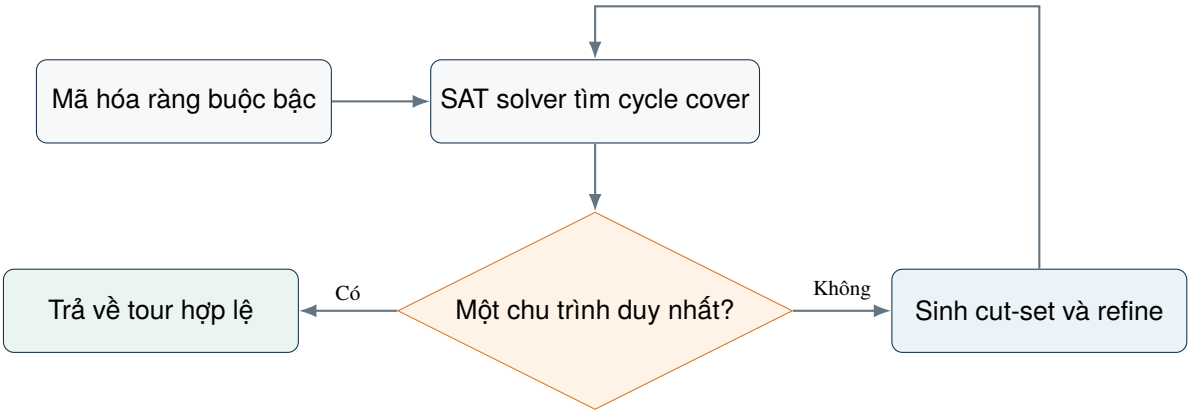
Với mỗi tập con $S \subset V$, về lý thuyết ta có thể viết một ràng buộc ngăn S trở thành chu trình riêng. Nhưng số tập con là cấp số mũ. Việc sinh toàn bộ trước khi giải có thể tạo công thức quá lớn.

Ý tưởng cốt lõi của CEGAR là: ban đầu chỉ đưa vào SAT phần dễ và nhỏ; khi solver cho một lời giải sai, ta mới bổ sung đúng ràng buộc cần thiết để loại kiểu sai đó.

4. CEGAR và cut-set bằng trực giác

4.1. CEGAR là vòng lặp "đoán, kiểm tra, sửa"

CEGAR là viết tắt của *Counterexample-Guided Abstraction Refinement*. Với HCP, quy trình có thể hiểu như sau:



Hình 4.1.: Vòng lặp CEGAR cho HCP.

4.2. Blocking subcycle và cut-set khác nhau thế nào?

Giả sử solver tạo ra một chu trình con trên tập đỉnh S.

Blocking subcycle nói: "Không được chọn lại chính xác các cạnh tạo ra chu trình con này." Cách này có thể chỉ loại một cấu hình rất cụ thể.

Cut-set nhìn rộng hơn. Gọi $\delta(S)$ là tập cạnh nối một đỉnh trong S với một đỉnh ngoài S. Một chu trình Hamilton phải đi ra và quay lại S, nên cần ít nhất hai cạnh qua biên:

$$\sum_{e \in \delta(S)} x_e \geq 2.$$

Ví dụ hòn đảo

Hãy hình dung S là một hòn đảo. Một tour đi qua toàn bộ bản đồ phải có đường vào và đường ra khỏi đảo. Cut-set không chỉ cấm một con đường nội bộ cụ thể; nó buộc lời giải phải kết nối đảo với bên ngoài.

4.3. 2-opt và 3-opt: sửa lời giải thay vì bỏ đi

Khi SAT trả về nhiều chu trình, ta không nhất thiết loại ngay lời giải đó. Nếu hai chu trình có các cạnh chéo thích hợp, 2-opt có thể tháo hai cạnh và nối lại để hợp nhất chúng. Restricted 3-opt thử một phép nối phức tạp hơn với ba cạnh.

Nếu sửa thành công, ta có lời giải mà không cần gọi SAT thêm. Nếu chỉ sửa được một phần, cycle cover đã cải thiện có thể cung cấp thông tin cho bước refine.

Patcher là heuristic: nó có thể giúp rất nhiều trên một họ graph nhưng tốn thời gian hoặc dẫn tìm kiếm về cùng một attractor trên họ khác. Vì vậy phương pháp mới không nên bật mọi patcher vô điều kiện.

5. Ta học được gì từ các baseline?

5.1. Phân biệt cấu hình đơn, portfolio và VBS

Ba khái niệm thường bị nhầm lẫn:

Khái niệm	Cách hoạt động	Có thể chạy như một solver thật?
Cấu hình đơn	Một bộ tham số cố định cho mọi instance.	Có.
Portfolio tĩnh	Thử nhiều cấu hình theo thứ tự cố định, dùng chung một ngân sách được quy định trước.	Có, nếu ngân sách tổng được tính công bằng.
VBS	Sau khi biết mọi kết quả, chọn hồi tố cấu hình tốt nhất cho từng instance.	Không. Đây là oracle và chỉ là cận trên.

5.2. Bức tranh dữ liệu hiện có

Dữ liệu per-instance trong repository cho thấy:

Kết quả	Số instance	Cách diễn giải
SAT-CEGAR tốt nhất, một cấu hình	957/1001	Mốc thực tế mạnh nhất trong các cột CEGAR đã ghi nhận.
Union của 16 biến thể SAT-CEGAR	963/1001	VBS trong họ SAT-CEGAR; chưa phải một solver đơn.
Union của 21 phương pháp có log đầy đủ	968/1001	Bao gồm các lời giải bổ sung của ASP và Picat.
Core chưa được 21 phương pháp giải	33/1001	Mục tiêu nghiên cứu quan trọng nhất.
HCup outside-mergecutset	937/1001	Tổng do bài HCup báo cáo; chưa có log per-instance cục bộ để tính overlap.
HCup hfp-preventsubcycle	917/1001	Cấu hình user propagator đơn mạnh nhất trong báo cáo HCup.
HCup VBS-propagator	961/1001	Oracle trên nhiều cấu hình, không phải một solver đơn.

Bảng 5.1.: Các mốc coverage cần phân biệt khi thiết kế phương pháp mới.

Không có một kỹ thuật đơn lẻ thắng trên mọi graph. Muốn vượt mốc 957 bằng một phương pháp thực, ta phải tận dụng tính bổ sung giữa encoding, CEGAR, inprocessing, model repair và user propagation, nhưng phải có cơ chế chọn đúng thời điểm.

5.3. Bài học cụ thể từ từng baseline

Nguồn	Kỹ thuật đáng học	Cách đưa vào phương pháp chung
CEGAR cũ	Incremental SAT và subtour elimination.	Là backbone bảo đảm tính đúng và tính đầy đủ.
CEGAR cut-set	Cut-set mạnh hơn blocking một cycle cụ thể; 2-loop, symmetry và 2-opt.	Dùng làm phase khởi đầu nhanh và làm ngôn ngữ chung cho cut pool.
CCaDical variants	Một cấu hình đơn đạt coverage rất cao.	Backend mặc định; không nên thay thế trước khi có bằng chứng paired.
Sinz variants	Có các unique win so với CCaDical.	Encoding thay thế khi telemetry cho thấy phase đầu bị hardening.
HCup outside-mergecutset	Merge cycle cover kết hợp inprocessing của CaDiCaL.	Chế độ CEGAR chính cho graph phù hợp, đặc biệt khi inprocessing có lợi.
HCup HFP	Prevent-subcycle can thiệp ngay trong quá trình CDCL.	Phase chuyên biệt cho merge stall và graph nhiều đỉnh bậc 2.
ASP	Biểu diễn connectivity/nogood toàn cục có tính bổ sung.	Chuyển ý tưởng đã kiểm chứng thành component hoặc reachability cuts trong SAT.
Picat	Có một unique win đã ghi nhận trên graph725.	Chưa đủ bằng chứng cơ chế; cần source và trace trước khi kết luận kỹ thuật nào tạo ra lợi ích.
HCP hiện tại	Nhiều prepass, patcher và decomposition.	Giữ dưới dạng module có gate; không bật toàn bộ mặc định.

6. Phương pháp đề xuất: Adaptive Hybrid SAT-HCP

6.1. Một phương pháp, không phải một túi solver

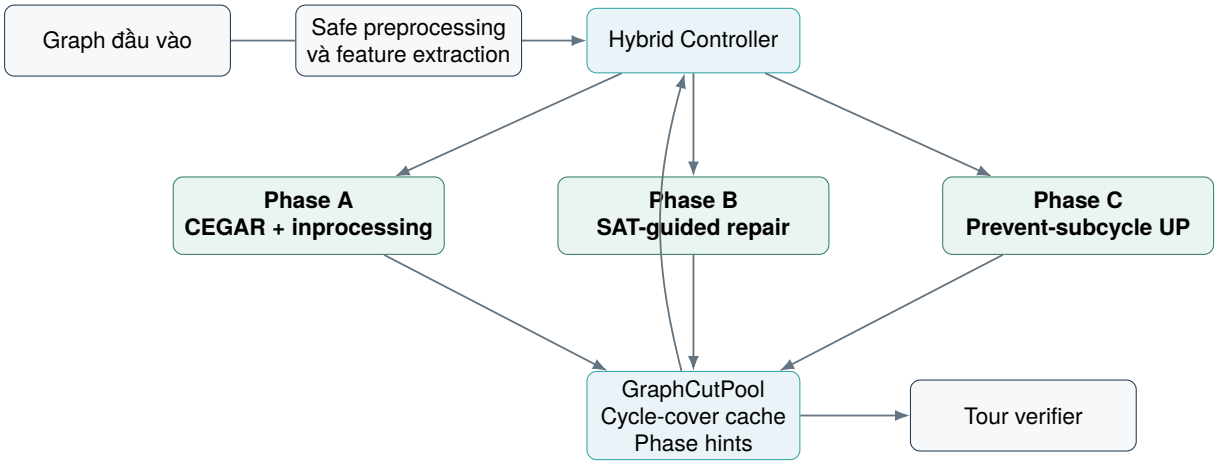
Phương pháp đề xuất có một executable và một controller cố định. Bên trong, nó có thể chuyển phase hoặc rebuild SAT state, nhưng mọi quyết định phải dựa trên đặc trưng graph và telemetry quan sát được trong lúc chạy, không dựa trên đáp án của test set.

Ta gọi phương pháp là **Adaptive Hybrid SAT-HCP**, viết tắt **AHS-HCP**.

6.2. Năm nguyên tắc thiết kế

1. **SAT là lõi chính xác:** mọi heuristic chỉ đề xuất hoặc sửa lời giải; tour cuối phải được kiểm chứng.
2. **Cut ở mức đồ thị là tri thức chung:** cut-set được lưu bằng đỉnh/cạnh, có thể mã hóa lại khi đổi encoding.
3. **Thích nghi có kiểm soát:** chỉ bật module khi tín hiệu cho thấy nó phù hợp.
4. **Tổng ngân sách là cố định:** mọi phase chia sẻ cùng giới hạn 1800 giây.
5. **Có thể giải thích:** log phải nói rõ vì sao controller chuyển phase.

6.3. Kiến trúc tổng thể



Hình 6.1.: Các backend không hoạt động độc lập: chúng trao đổi tri thức qua cut pool và được điều phối bởi một controller duy nhất.

6.4. Phase A: CEGAR có inprocessing

Phase A bắt đầu bằng cấu hình SAT-CEGAR mạnh nhất đã biết. CaDiCaL được phép dùng inprocessing để rút gọn công thức và cấu trúc tìm kiếm.

Mỗi cycle cover được xử lý theo thứ tự:

1. kiểm tra có phải một chu trình Hamilton hay không;
2. thử 2-opt hoặc restricted 3-opt khi xác suất merge cao;
3. sinh cut-set cho các component còn lại;
4. ghi telemetry rồi gọi SAT tiếp.

6.5. Phase B: SAT-guided repair

Thay vì để patcher heuristic thử vô hạn, Phase B biến việc sửa cycle cover thành một bài toán SAT nhỏ hơn:

- giữ các cạnh "tốt" bằng assumptions;
- chỉ mở các cạnh quanh biên giữa các subcycle;
- nếu bài toán nhỏ UNSAT, dùng thông tin thất bại để mở rộng vùng sửa;
- nếu tìm được tour, kiểm chứng trực tiếp trên graph gốc.

Đây là một dạng large-neighborhood search nhưng lõi tìm kiếm vẫn là SAT.

6.6. Phase C: prevent-subcycle user propagation

User propagator theo dõi assignment chưa hoàn chỉnh. Nếu một cạnh sắp đóng một chu trình nhỏ trước khi chứa tất cả đỉnh, propagator có thể đẩy literal tương ứng về false và cung cấp lý do cho solver.

Trực giác

CEGAR thường đợi SAT solver dựng xong một cycle cover sai rồi mới sửa. User propagation giống như người giám sát phát hiện vòng nhỏ đang sắp khép lại và ngăn nó ngay trong lúc xây dựng.

6.7. Vì sao cần rebuild giữa Phase A và Phase C?

Inprocessing có thể đổi, loại hoặc gộp biến nội bộ. User propagator lại cần ánh xạ ổn định giữa biến SAT và cạnh graph. Do đó hai lợi ích này không luôn tương thích trong cùng một state.

Giải pháp đề xuất:

1. kết thúc Phase A tại một điểm chuyển an toàn;
2. xuất mọi graph cut độc lập encoding;
3. tạo SAT state mới ở chế độ tương thích IPASIR-UP;
4. mã hóa lại degree constraints và import graph cuts;
5. dùng cycle cover tốt nhất làm phase hint.

Learned clause nội bộ không phải lúc nào chuyển được, nhưng cut-set ở mức graph thì chuyển được.

7. Hybrid Controller quyết định như thế nào?

7.1. Telemetry tối thiểu

Sau mỗi vòng CEGAR, controller ghi:

- số subcycle và phân bố kích thước;
- số subcycle được 2-opt/3-opt hợp nhất;
- số vòng liên tiếp không giảm cycle count;
- thời gian SAT call gần nhất và tỷ lệ tăng;
- số cut-set, độ dài cut và số blocking clause;
- thời gian dành cho từng patcher;
- graph density m/n, tỷ lệ đỉnh bậc 2, max degree và số hub.

7.2. Controller luật trước, học máy sau

Phiên bản đầu nên dùng luật giải thích được:

Tín hiệu	Diễn giải khả dĩ	Hành động
$m/n \geq 3$, nhiều hub	Dense graph; SAT branching và clause size có thể là nút thắt.	Duy trì inprocessing, ưu tiên merge-cutset; chỉ bật decomposition sau khi có stall.
Nhiều đỉnh bậc 2, merge progress bằng 0	Cycle cover dễ rơi vào các vòng nhỏ cưỡng bức.	Chuyển sang prevent-subcycle UP hoặc connectivity cuts mạnh hơn.
Ít subcycle lớn, patcher liên tục giảm cycle count	Lời giải đang gần một tour.	Cho repair thêm ngân sách; chưa rebuild solver.
Nhiều subcycle nhỏ lặp lại	Patcher có thể quay lại cùng attractor.	Giảm patching, thêm cut sớm và đa dạng phase.
SAT call cuối chiếm phần lớn runtime	State có dấu hiệu hardening.	Export cut pool rồi rebuild với encoding/phase khác.

Bảng 7.1.: Các luật controller ban đầu. Ngưỡng cụ thể phải được chọn trên training set.

7.3. Phân bổ ngân sách

Không nên cho mỗi backend 1800 giây rồi cộng kết quả, vì như vậy phương pháp hybrid được hưởng nhiều tài nguyên hơn baseline. Một cách công bằng là:

$$T_{\text{preprocess}} + T_A + T_B + T_C \leq 1800 \text{ giây}.$$

Controller có thể chuyển phase theo sự kiện thay vì một mốc thời gian cứng. Ví dụ, Phase A tiếp tục khi vẫn tạo tiến bộ; nó chuyển sớm nếu nhiều vòng liên tiếp không cải thiện và SAT call tăng nhanh.

Không được nhìn tên graph rồi tra bảng “solver nào từng thắng”. Quy tắc phải dựa trên đặc trưng tính được từ graph hoặc telemetry sinh ra trong lần chạy hiện tại. Nếu không, kết quả chỉ là memorization của benchmark.

8. Các hướng brainstorming có tiềm năng

8.1. Encoding lai theo từng đỉnh

Không nhất thiết mọi đỉnh phải dùng cùng một cardinality encoding:

- bậc thấp: pairwise hoặc binomial, ít biến phụ;
- bậc trung bình: Sinz, lan truyền ổn định;
- hub bậc cao: commander hoặc encoding phân cấp.

Đây là một công thức SAT duy nhất nhưng được thiết kế theo cấu trúc cục bộ của graph.

8.2. GraphCutPool độc lập encoding

Mỗi cut được lưu ở dạng cấu trúc:

$$(S,\ \delta(S),\ \text{nguồn},\ \text{số lần hữu ích},\ \text{thời điểm tạo}).$$

Khi rebuild, cut được dịch sang biến của encoding mới. Có thể xếp hạng cut theo độ ngắn, số lần propagation hoặc mức độ tái xuất hiện của component.

8.3. Phase hints từ cycle cover tốt nhất

Khi tạo SAT state mới:

- ưu tiên true cho cạnh thuộc cycle cover tốt nhất;
- ưu tiên false cho cạnh nhiều lần đóng subcycle xấu;
- giảm dần độ tin cậy sau mỗi lần thất bại để tránh khóa vào attractor.

8.4. Tổng quát hóa cut theo xung đột

Nếu nhiều cycle cover khác nhau liên tục tạo cùng một phân hoạch component, ta có thể tổng quát hóa các blocking clause riêng lẻ thành một connectivity cut mạnh hơn. Đây là nơi có thể hấp thụ trực giác về nogood toàn cục từ ASP mà vẫn giữ backend SAT.

8.5. Học controller bằng dữ liệu

Sau khi có log đầy đủ, một mô hình nhỏ có thể dự đoán:

- encoding khởi đầu;

- có nên bật user propagation;
- thời điểm chuyển phase;
- patcher nào đáng chạy;
- cách chia ngân sách.

Tuy nhiên, 1001 instance là tập dữ liệu nhỏ và chứa nhiều graph cùng họ. Phải chia train/test theo *họ cấu trúc*, không chia ngẫu nhiên từng graph. Nếu graph950 được train còn graph963 gần như cùng cấu trúc nằm trong test, kết quả có thể bị leakage.

9. Thiết kế thực nghiệm công bằng

9.1. Câu hỏi nghiên cứu

RQ1. Một controller đơn có vượt cấu hình SAT-CEGAR đơn tốt nhất 957/1001 không?

RQ2. Cut transfer giữa phase có tạo unique wins hay chỉ thêm overhead?

RQ3. Prevent-subcycle UP có bổ sung cho CEGAR inprocessing trên core-33 không?

RQ4. SAT-guided repair có thắng 2-opt/3-opt thuần trên nhóm merge stall không?

RQ5. Encoding lai theo đỉnh có giảm runtime mà không gây regression trên easy set không?

9.2. Ba mức kết quả phải báo cáo riêng

1. **Single static SAT:** cấu hình cố định, mốc tham chiếu 957.
2. **Adaptive single-method SAT:** AHS-HCP, một executable và một ngân sách tổng.
3. **Unrestricted portfolio:** có thể bao gồm ASP/Picat, dùng để đo coverage thực dụng tối đa.

9.3. Tập đánh giá trọng tâm

Ngoài full set 1001 graph, cần theo dõi:

- **Core-33:** chưa được 21 phương pháp có log đầy đủ giải.
- **SAT-only open-38:** core-33 cộng graph677, graph725, graph744, graph810 và graph940, vốn đang được ASP/Picat bổ sung.
- **Easy-overhead set:** kiểm tra phương pháp thích nghi không làm graph dễ chậm đi quá nhiều.
- **Các họ cấu trúc:** dense B1, merge-stall B2a, long-call B2b và attractor A.

9.4. Chỉ số đánh giá

Chỉ số	Ý nghĩa
Solved count	Số instance có tour hợp lệ trong cap. Đây là chỉ số chính.
PAR-2	Runtime nếu giải được; phạt 2× timeout nếu không giải. Phản ánh cả tốc độ và độ bền.
Median/p90	Tốc độ trên phần instance đã giải. Không thay thế solved count.
Unique wins	Instance chỉ phương pháp này giải được; cho thấy tính bổ sung thực.
Marginal gain	Số instance một phase bổ sung sau các phase trước.
Phase overhead	Thời gian controller, rebuild, encoding, patching và verification.

9.5. Ablation bắt buộc

Cấu hình	Cut pool	Repair SAT	UP	Controller
CEGAR base	×	×	×	×
+ cut transfer	✓	×	×	×
+ SAT-guided repair	✓	✓	×	×
+ user propagation	✓	✓	✓	×
AHS-HCP đầy đủ	✓	✓	✓	✓

Bảng 9.1.: Ablation giúp xác định module nào thực sự tạo lợi ích.

Phương pháp mới phải được chạy như một cấu hình đã chốt, trên cùng máy, cùng giới hạn CPU/RAM, cùng verifier và cùng cách tính timeout. Không so runtime trực tiếp giữa các CSV được đo trên phần cứng khác nhau.

10. Lộ trình triển khai

10.1. Giai đoạn 0: hoàn thiện bằng chứng

1. Chạy hai cấu hình HCup mạnh nhất trên core-33.
2. Import JSONL vào bảng coverage động.
3. Xác định graph nào HCup bổ sung cho SAT-CEGAR.
4. Thu telemetry thống nhất cho CEGAR và HCup.

10.2. Giai đoạn 1: GraphCutPool và telemetry

Đây là bước rủi ro thấp nhất và có giá trị độc lập:

- định nghĩa cut ở mức graph;
- gắn provenance và thống kê sử dụng;
- xuất/import cut khi rebuild;
- bảo đảm mọi cut đều sound bằng unit test trên graph nhỏ.

10.3. Giai đoạn 2: Hybrid Controller luật

Controller ban đầu chỉ cần một state machine nhỏ:

1. `FAST_CEGAR`;
2. `SAT_REPAIR`;
3. `UP_REBUILD`;
4. `FINAL_FALLBACK`;
5. `SOLVED` hoặc `TIMEOUT`.

Mọi chuyển state phải ghi event, lý do, thời gian còn lại và snapshot telemetry.

10.4. Giai đoạn 3: tích hợp kỹ thuật HCup

Có hai mức tích hợp:

Mức thực nghiệm nhanh. Một driver cố định điều phối hai binary và dùng chung file cut/cycle-cover. Dễ kiểm chứng ý tưởng nhưng vẫn có overhead process.

Mức phương pháp hoàn chỉnh. Port prevent-subcycle hoặc đóng gói HCup thành library có API ổn định. Khi đó controller, cut pool và verifier nằm trong một executable.

Nên chứng minh tính bổ sung ở mức driver trước, rồi mới trả chi phí port sâu.

10.5. Giai đoạn 4: cải tiến mới và học selector

Chỉ sau khi controller luật ổn định mới thử encoding lai, SAT-guided LNS và model học. Mỗi ý tưởng cần một ablation riêng; nếu gộp nhiều thay đổi cùng lúc, ta không biết nguyên nhân của thắng hoặc thua.

11. Pseudocode dễ đọc

Thuật toán AHS-HCP

```
INPUT: graph G, total budget T
OUTPUT: verified Hamiltonian cycle, or UNKNOWN

1. preprocess G using only sound reductions
2. compute structural features
3. create GraphCutPool and HybridController
4. start Phase A: CEGAR with inprocessing
5. while remaining time > 0:
   a. ask SAT for a candidate cycle cover
   b. if candidate is one cycle: verify and return it
   c. try the repair selected by Controller
   d. if repair creates one cycle: verify and return it
   e. generate sound graph cuts and store them
   f. update telemetry
   g. if Controller requests a phase change:
      - export graph cuts and best cycle cover
      - rebuild the selected SAT mode
      - import cuts and phase hints
6. return UNKNOWN
```

12. Câu hỏi thường gặp

Phương pháp này có còn là SAT-only không?

Có, nếu ASP và Picat chỉ được dùng làm nguồn ý tưởng hoặc baseline, còn mọi phase tìm kiếm của AHS-HCP đều dựa trên SAT/CaDiCaL, cut-set và user propagation.

Rebuild solver có làm mất hết công sức cũ không?

Một phần learned clause nội bộ có thể mất, nhưng graph cuts, cycle cover tốt nhất, phase hints và telemetry vẫn được giữ. Mục tiêu nghiên cứu là chứng minh phần tri thức chuyển được bù lại chi phí rebuild.

Tại sao không chạy CEGAR và HCup song song?

Chạy song song có thể tăng coverage thực dụng, nhưng dùng nhiều CPU hơn baseline. Đây có thể là một chế độ portfolio đa lõi riêng, không nên dùng làm kết quả chính cho phương pháp một lõi.

Có nên dùng học máy ngay không?

Chưa. Trước hết cần log đầy đủ và controller luật làm baseline. Nếu học máy được thêm quá sớm, rất khó phân biệt lợi ích thật với overfitting vào 1001 graph.

Nếu trả về SAT thì đã chắc chắn đúng chưa?

Chưa. Luôn phải kiểm tra rằng tour chứa đúng mọi đỉnh một lần và mọi cặp liên tiếp là cạnh hợp lệ, kể cả cạnh quay về điểm đầu.

A. Bảng thuật ngữ

Thuật ngữ	Giải thích ngắn
Assignment	Một cách gán đúng/sai cho các biến SAT.
Baseline	Phương pháp tham chiếu dùng để so sánh.
Blocking clause	Clause được thêm để ngăn solver lặp lại một cấu hình sai.
CaDiCaL	SAT solver CDCL hiện đại được dùng trong các phương pháp đang xét.
CEGAR	Vòng lặp giải abstraction, tìm counterexample rồi refine.
Clause	Phép hoặc của các literal.
CNF	Phép và của nhiều clause.
Cut-set	Tập cạnh đi qua biên của một tập đỉnh.
Cycle cover	Tập nhiều chu trình rời nhau phủ toàn bộ đỉnh.
Encoding	Cách chuyển ràng buộc bài toán sang clause SAT.
Inprocessing	Các phép rút gọn công thức được SAT solver thực hiện trong lúc giải.
IPASIR-UP	Giao diện cho phép user propagator tương tác với SAT solver.
Literal	Một biến hoặc phủ định của biến.
PAR-2	Chỉ số runtime phạt timeout bằng hai lần cap.
Phase hint	Gợi ý giá trị đúng/sai ưu tiên khi solver bắt đầu tìm kiếm.
Subcycle	Chu trình chỉ chứa một phần đỉnh của graph.
Telemetry	Dữ liệu đo động lực tìm kiếm trong lúc chạy.
User propagator	Module ngoài cung cấp propagation hoặc clause chuyên biệt cho bài toán.
VBS	Virtual Best Solver, oracle chọn kết quả tốt nhất sau khi đã biết mọi run.

B. Tái tạo báo cáo coverage và benchmark

Biên dịch chính tài liệu này bằng XeLaTeX:

```
mkdir -p output/pdf
latexmk -xelatex -interaction=nonstopmode \
  -outdir=output/pdf docs/adaptive-hybrid-sat-hcp.tex
```

Sinh lại các CSV coverage:

```
python3 tools/build_hybrid_coverage.py
```

Chạy hai cấu hình HCup mặc định trên core-33:

```
python3 tools/hcp_benchmark.py \
  --config benchmarks/hybrid_variants.json \
  --set historical_open_33 \
  --stop-on-solved \
  --output benchmark-runs/hcup-open33
```

Import kết quả mới vào coverage động:

```
python3 tools/build_hybrid_coverage.py \
  --benchmark-results benchmark-runs/hcup-open33/results.jsonl
```

C. Tài liệu tham khảo

Tài liệu tham khảo

[1] R. Ohashi, T. Soh, D. Le Berre, H. Nabeshima, M. Banbara, K. Inoue, và N. Tamura. *SAT-Based CEGAR Method for the Hamiltonian Cycle Problem Enhanced by Cut-Set Constraints*. SAT 2025, LIPIcs 341, Article 24, 2025.

[2] F. Funder. *An UPgrade for HCP: Solving the Hamiltonian Cycle Problem via User Propagators*. Diplomarbeit, Technische Universität Wien, 2026.

[3] A. Biere, M. Heule, H. van Maaren, và T. Walsh, biên tập. *Handbook of Satisfiability*. IOS Press, 2009.

[4] R. M. Karp. Reducibility among combinatorial problems. Trong *Complexity of Computer Computations*, 1972.

Tài liệu này đi kèm mã nguồn và pipeline kiểm chứng trong repository HCP-CEGAR.